Glimmerfen en Producción

Reporte del Proyecto Final — explicado paso a paso

Deep Learning y Sistemas Inteligentes · Universidad del Valle · 2026
Equipo: Cindy Gualim y Gadiel Ocaña
Modelo base: Proyecto 2 — Diseño generativo de personajes (GAN)

Este documento explica, en lenguaje simple, qué se construyó, por qué, cómo funciona, qué tan bien funciona (con números y evidencia visual), y qué falta por hacer antes de la entrega del 15 de noviembre.

1. ¿Qué es este proyecto, en una frase?

Tomamos una red neuronal generativa (una GAN) que Cindy y Gadiel ya habían entrenado en el Proyecto 2 — capaz de inventar personajes nuevos de un universo de fantasía llamado **Glimmerfen** — y la convertimos en un **servicio real**: una página web donde cualquier persona, sin saber nada de redes neuronales ni tener el código, puede pedir un personaje nuevo y recibirlo al instante. Esa es la parte que pide el enunciado: "un modelo está en producción si una persona ajena al equipo puede usarlo".

Además, no nos quedamos con el modelo tal cual: lo **mejoramos**. El modelo original solo generaba personajes al azar. Nosotros le enseñamos a generar un personaje **del elemento que se le pida** (fuego, agua, tierra, aire, luz o sombra), lo que lo hace mucho más útil para un caso de uso real.

¿Qué es una GAN? (explicación sin matemáticas)

Una GAN (*Generative Adversarial Network*, "red generativa adversaria") son en realidad **dos** redes neuronales compitiendo entre sí, como un falsificador contra un detective:

- El **Generador** recibe puro ruido aleatorio (una lista de 100 números al azar) y trata de convertirlo en una imagen que parezca un personaje real.
- El **Discriminador** mira imágenes (algunas reales del dataset, otras inventadas por el Generador) y trata de adivinar cuáles son falsas.
- Ambos entrenan a la vez: el Generador mejora para engañar al Discriminador, y el Discriminador mejora para no dejarse engañar. Tras miles de rondas, el Generador queda tan bueno que produce imágenes convincentes de personajes que **nunca existieron**.

Una vez entrenado, solo se usa el Generador (el Discriminador se descarta): se le da ruido nuevo y devuelve un personaje nuevo. Eso es literalmente lo único que hace nuestro servicio en producción.

2. El problema que resuelve (sección 1 del enunciado)

Usuario: un estudio indie (o un creador individual) que está diseñando nuevas criaturas para un juego o cómic de fantasía.

Problema: bocetar variantes de personajes a mano, una por una, es lento y costoso en la fase temprana de ideación — antes de saber siquiera si un diseño vale la pena pulirlo.

Decisión que depende del modelo: qué bocetos generados (y de qué elemento) pasan de "candidato generado" a "encargo de arte final a un ilustrador humano". El modelo acelera el embudo creativo — no reemplaza al ilustrador, filtra en qué conviene invertir su tiempo.

3. De dónde viene el modelo (Proyecto de origen)

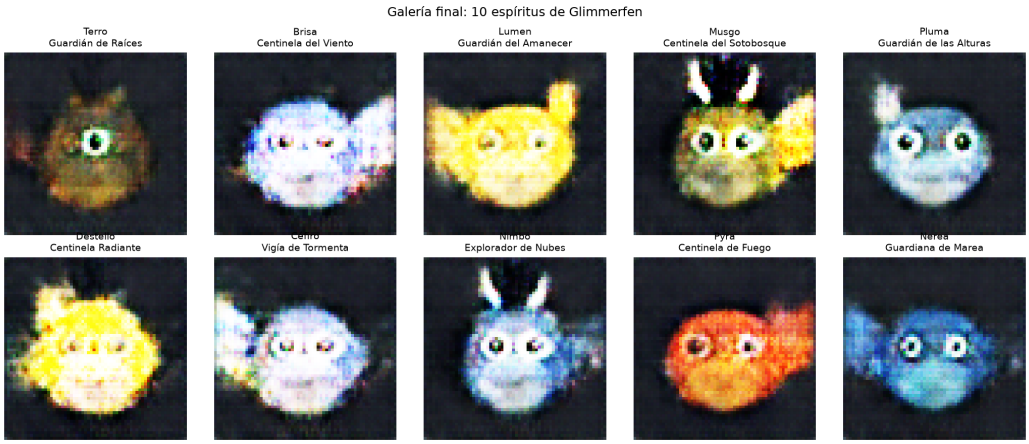
El modelo no se entrenó desde cero para este proyecto — el enunciado permite (y pide) reusar un modelo de los Proyectos 1, 2 o 3. Se usó el **Proyecto 2**: una DCGAN entrenada sobre un dataset

procedural (generado por código propio con la librería PIL, no descargado de internet), con 4000 imágenes de 64×64 píxeles de "espíritus guardianes" de un bosque, cada uno de uno de 6 elementos.



Muestra del dataset de entrenamiento: personajes generados por código, no arte de terceros.

En el Proyecto 2 se probaron tres variantes de entrenamiento (la básica, una con una función de pérdida distinta — WGAN-GP —, y una con una técnica de estabilización llamada normalización espectral) y se comparó cuál generaba personajes más consistentes. Ganó la variante con **normalización espectral** (stab_spectral_norm), y es la que usamos como punto de partida ("Fase 1") de este proyecto.



Los 10 personajes finales del Proyecto 2, generados por el modelo base usado aquí.

4. La mejora: de "al azar" a "a pedido" (Fase 2)

El enunciado permite mejorar el modelo ("puede mejorar el modelo; no se pide uno nuevo"). El modelo original tiene una limitación importante para el caso de uso real: no se puede pedir un elemento específico, solo se genera uno al azar. Si el estudio quiere específicamente un personaje de **fuego** para su nuevo nivel de volcán, con el modelo original tocaría generar personajes al azar y esperar que, por suerte, salga uno de fuego.

Para arreglar esto se construyó una **Conditional GAN** ("GAN condicional"): se le enseña al Generador y al Discriminador a prestar atención a una etiqueta extra (el elemento: fuego/agua/tierra/aire/luz/sombra), no solo al ruido. En términos simples: además del ruido aleatorio, ahora también se le dice "y que sea de fuego", y la red aprendió a obedecer esa instrucción.

Cómo se hizo (sin tecnicismos)

- El dataset ya sabía, por dentro, de qué elemento era cada personaje (se generan con una paleta de color por elemento) — pero esa información nunca se guardaba. Se regeneró el mismo dataset (con la misma semilla, o sea exactamente las mismas 4000 imágenes) guardando también esa etiqueta.
- Al Generador se le agregó una pequeña tabla de "traducción" (un *embedding*) que convierte el nombre del elemento en un grupo de números, y esos números se mezclan con el ruido antes de generar la imagen.
- Al Discriminador se le agregó lo mismo, para que también aprenda a penalizar una imagen que no combine con el elemento que se le dijo que tenía.
- Se reentrenó con la misma receta que ya había funcionado bien en el Proyecto 2 (30 épocas, ~5 minutos en la Mac del equipo).

5. ¿Funcionó? — Validación de la hipótesis

Antes de entrenar se definió, por escrito, cómo se iba a medir el éxito (esto es clave para la sección 5 del enunciado: el criterio se fija **antes** de ver resultados, para no "acomodar" la conclusión después).

Hipótesis: condicionar la GAN al elemento permite pedir un elemento específico sin perder la diversidad visual lograda en el Proyecto 2.

Métrica: de 100 personajes generados pidiendo el elemento X, ¿qué porcentaje realmente tiene el elemento X como color dominante? (medido con un clasificador de color automático, no es el modelo juzgándose a sí mismo).

Criterio de éxito (fijado antes de entrenar): acertar el elemento pedido al menos 80% de las veces.

Línea base de comparación: el modelo viejo (sin poder elegir), que acierta el elemento por pura casualidad ~16.7% de las veces (1 de 6 elementos).

Resultado medido

Elemento	Acierto condicional	Frecuencia natural (línea base)
fuego	100%	15.7%
agua	100%	9.5%
tierra	100%	27.0%
aire	100%	18.8%
luz	100%	13.8%
sombra	100%	15.2%
Promedio	**100%**	**16.7%**

100% de acierto en los 6 elementos, muy por encima del 80% exigido y del ~16.7% de la línea base. La hipótesis se confirma con margen amplio.



4 personajes generados por fila, pidiendo un elemento distinto en cada fila (de arriba a abajo: fuego, agua, tierra, aire, luz, sombra). El color de cada fila es consistente con lo pedido.

La parte honesta: lo que no salió gratis

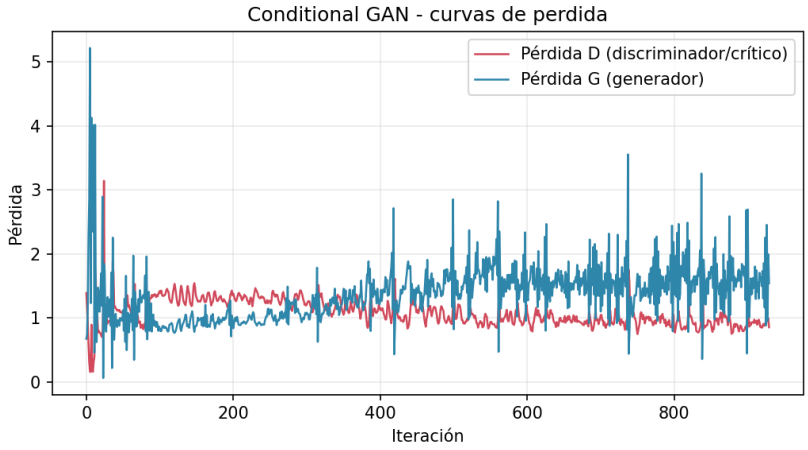
Al medir con más cuidado, se encontró algo que **no estaba en el criterio de éxito original**, pero que es importante contarlo en vez de ocultarlo (igual que el equipo ya hizo en el Proyecto 2 al documentar los problemas de memoria con WGAN-GP): los personajes generados **dentro de un mismo elemento se parecen mucho entre sí** — casi la misma pose y forma, variando sobre todo el tono exacto de color. Se midió la "diversidad" comparando qué tan diferentes son, en promedio, las imágenes entre sí dentro del mismo elemento:

Elemento	Diversidad condicional (MSE)	Diversidad incondicional (MSE)
fuego	12.8	~1895
agua	18.8	~1874
tierra	18.8	~1693
aire	22.7	~2736
luz	12.6	~2691
sombra	14.8	~1625

Mientras más alto el número, más diferentes son las imágenes entre sí. El modelo condicional (segunda columna) es ~100 veces menos diverso dentro de cada elemento que el modelo original agrupado por color (tercera columna). En otras palabras: **ganamos control total del elemento, a costa de variedad dentro de ese elemento**. Es un trade-off real y conocido en GANs condicionales (se puede mejorar con más épocas o técnicas anti-colapso, pero eso queda fuera del alcance de este curso).

¿El entrenamiento fue estable?

Sí. Ninguna de las dos redes "ganó" por completo a la otra (si eso pasa, la GAN deja de aprender). Las curvas de pérdida se mantienen en un rango razonable durante todo el entrenamiento:



Pérdida del Discriminador (rojo) y del Generador (azul) durante las 30 épocas de entrenamiento de la Conditional GAN.

6. Cómo se puso en producción (arquitectura)

"Poner en producción" aquí significa exactamente lo que pide el enunciado: que alguien sin el código, sin saber de IA, pueda usar el modelo desde su navegador. El diseño es deliberadamente simple porque el modelo es pequeño y no necesita tarjeta gráfica para usarse (solo para entrenarse):

1. **Entrenamiento** (una sola vez, no en producción): se corre en la laptop del equipo, con Apple Silicon (GPU "MPS"), y produce un archivo de pesos (checkpoint.pt, ~27MB).
2. **Servicio** (app.py): un programa en Python con la librería **Gradio** que carga ese archivo una sola vez al arrancar, y expone una página web con un botón "Generar personaje", un campo de semilla y un menú para elegir el elemento.
3. **Despliegue**: **Hugging Face Spaces**, un servicio gratuito pensado justo para esto — aloja la app, le da una URL pública, y la corre en CPU (suficiente, porque generar una imagen toma milisegundos).
4. **Persona externa**: abre la URL, elige un elemento, hace clic, y recibe un personaje nuevo generado en el momento — ese es el "pase a producción" que pide la sección 7 del enunciado.

Por qué CPU y no GPU: el Generador tiene solo 5 capas y pesa ~27MB. Generar una sola imagen de 64×64 píxeles toma milisegundos en cualquier computadora normal — usar GPU en producción sería gastar recursos que no hacen falta.

7. Estado actual y lo que falta

Todo lo que depende del modelo (entrenarlo, mejorarlo, validarlo y servirlo localmente) está terminado y verificado con pruebas automáticas. Lo que falta son pasos manuales que solo el equipo puede hacer (crear cuentas, decidir con el docente, ensayar):

Tarea	Estado
Empaquetar el modelo entrenado (Fase 1)	**Listo**
Servicio web local (Gradio) funcionando	**Listo**
Mejora del modelo: Conditional GAN (Fase 2)	**Listo**
Entrenamiento de la Conditional GAN	**Listo**
Validación de hipótesis (sección 5)	**Listo**
app.py sirviendo el modelo condicional	**Listo**
Pruebas unitarias (46 tests)	**Listo**
README con las 7 secciones del enunciado	**Listo**
Despliegue público (Hugging Face Spaces)	**Pendiente**

Capturas / evidencias de uso real (sección 7)	**Pendiente**
Confirmar con el docente: dupla vs. trío	**Pendiente**
Ensayo de la presentación	**Pendiente**

Dos decisiones que son del equipo, no del código

• El enunciado pide **tríos**; este equipo quedó como **dupla** (mismas dos personas del Proyecto 2). Hay que confirmarlo con el docente antes de la presentación.

• Falta crear la cuenta en Hugging Face y publicar el Space para tener el enlace público — es la única pieza que falta para completar la sección 7 (evidencias) del README.

8. Cómo correrlo (para quien quiera probarlo)

Todo el código y los pesos ya entrenados están en el repositorio. No hace falta reentrenar nada para probarlo:

1. pip install -r requirements.txt
2. python app.py
3. Abrir http://127.0.0.1:7860 en el navegador.

El detalle completo (incluyendo cómo reentrenar la Conditional GAN desde cero, ~5 minutos) está en el README.md del repositorio, junto con las 7 secciones que pide el enunciado del curso.

Este reporte fue generado como apoyo para entender el proyecto de forma rápida; el documento técnico oficial del curso es el README.md del repositorio.